

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТУ “ЛЬВІВСЬКА
ПОЛІТЕХНІКА”**

Кафедра систем штучного інтелекту

Лабораторна робота №2
з дисципліни
“Середовище та компоненти у моделюванні та аналізі даних”

Виконав:
ст. гр. ШІ-31
Хомишин Д.М.

Викладач:
Бенч А. Я.

Тема: Побудова CI-конвеєра із використанням GitHub Actions.

Мета: Налаштувати CI: автоматичний запуск модульних задач (тести/лінт/збірка/перевірки) при push/pull_request; Налаштувати CD (умовно): публікація артефактів (логів, звітів, графіків) та/або деплой результатів (наприклад, у GitHub Pages) після успішного CI; . Навчитись запускати pipeline

Результати

Посилання на репозиторій: <https://github.com/DanyloShi/open-data-ai-analytics>

1. Налаштувати CI

```
name: CI Modules

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]
  workflow_dispatch:
    inputs:
      module:
        description: "Which module to run (or all)"
        required: true
        default: "all"
        type: choice
        options:
          - all
          - data_load
          - data_quality_analysis
          - data_research
          - visualization

permissions:
  contents: read

jobs:
  run-modules:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        module: [data_load, data_quality_analysis, data_research, visualization]

    steps:
      - name: Checkout
        uses: actions/checkout@v4
```

```

- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.12"
    cache: "pip"
    cache-dependency-path: |
      visualization/requirements.txt
      web/requirements.txt

- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    python -m pip install -r visualization/requirements.txt
    python -m pip install -r web/requirements.txt

- name: Run selected module pipeline
  if: ${{ github.event_name != 'workflow_dispatch' ||
github.event.inputs.module == 'all' || github.event.inputs.module == matrix.module
}}
  env:
    MODULE: ${{ matrix.module }}
  run: |
    set -euo pipefail
    mkdir -p artifacts/${MODULE}/work

    export WORK_DIR="$PWD/artifacts/${MODULE}/work"
    export DB_PATH="$WORK_DIR/db/air_quality.db"
    export REPORTS_DIR="$WORK_DIR/reports"
    export PLOTS_DIR="$WORK_DIR/plots"
    export CSV_PATH="$PWD/data/dataset.csv"

    python data_load/app.py | tee "artifacts/${MODULE}/data_load.log"

    if [ "${MODULE}" != "data_load" ]; then
      python "${MODULE}/app.py" | tee "artifacts/${MODULE}/${MODULE}.log"
    fi

- name: Collect generated outputs
  if: ${{ github.event_name != 'workflow_dispatch' ||
github.event.inputs.module == 'all' || github.event.inputs.module == matrix.module
}}
  env:
    MODULE: ${{ matrix.module }}
  run: |
    set -euo pipefail
    if [ -d "artifacts/${MODULE}/work/reports" ]; then

```

```

        cp -R "artifacts/${MODULE}/work/reports" "artifacts/${MODULE}/reports"
    fi

    if [ -d "artifacts/${MODULE}/work/plots" ]; then
        cp -R "artifacts/${MODULE}/work/plots" "artifacts/${MODULE}/plots"
    fi

-   name: Upload artifacts
        if: ${{ github.event_name != 'workflow_dispatch' ||
github.event.inputs.module == 'all' || github.event.inputs.module == matrix.module
}}

    uses: actions/upload-artifact@v4
    with:
        name: artifacts-${{ matrix.module }}-${{ github.run_id }}
        path: artifacts/${{ matrix.module }}
        retention-days: 14

```

Було створено workflow GitHub Actions для CI-конвеєра, який спрацює при push, pull request та вручну. Проміжні результати кожного етапу зберігаються в artifacts.

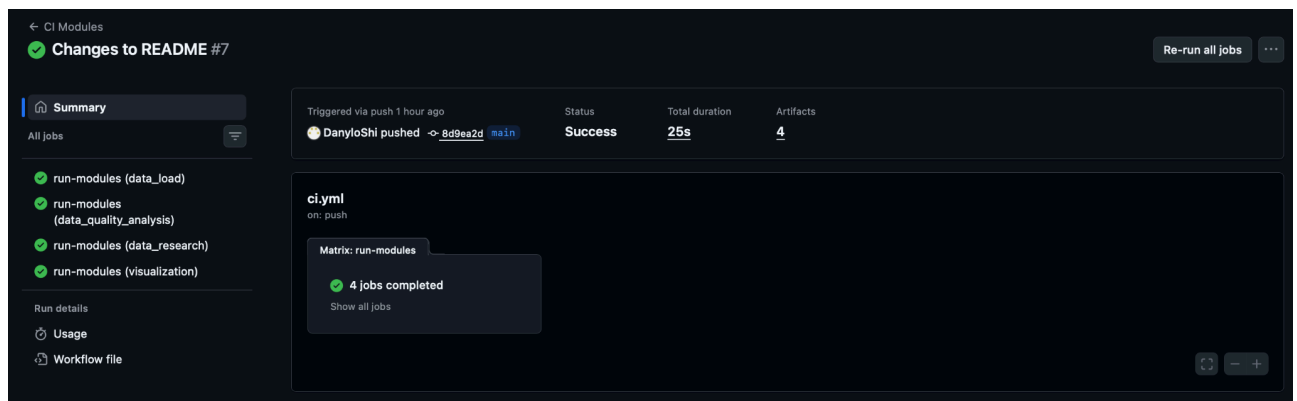


Рис. 1. Виконання CI-конвеєра

2. Налаштувати CD

Для збереження графіків у файлі ci.yml було додано наступний код:

```

-   name: Collect generated outputs
        if: ${{ github.event_name != 'workflow_dispatch' ||
github.event.inputs.module == 'all' || github.event.inputs.module == matrix.module }}
    env:
        MODULE: ${{ matrix.module }}
    run: |
        set -euo pipefail
        if [ -d "artifacts/${MODULE}/work/reports" ]; then
            cp -R "artifacts/${MODULE}/work/reports" "artifacts/${MODULE}/reports"
        fi

```

```

        if [ -d "artifacts/${MODULE}/work/plots" ]; then
            cp -R "artifacts/${MODULE}/work/plots" "artifacts/${MODULE}/plots"
        fi

- name: Upload artifacts
  if: ${ github.event_name != 'workflow_dispatch' || github.event.inputs.module ==
'all' || github.event.inputs.module == matrix.module }
  uses: actions/upload-artifact@v4
  with:
    name: artifacts-${ github.run_id }-${ matrix.module }
    path: artifacts/${ matrix.module }
    retention-days: 14

```

3. Self-hosted runner

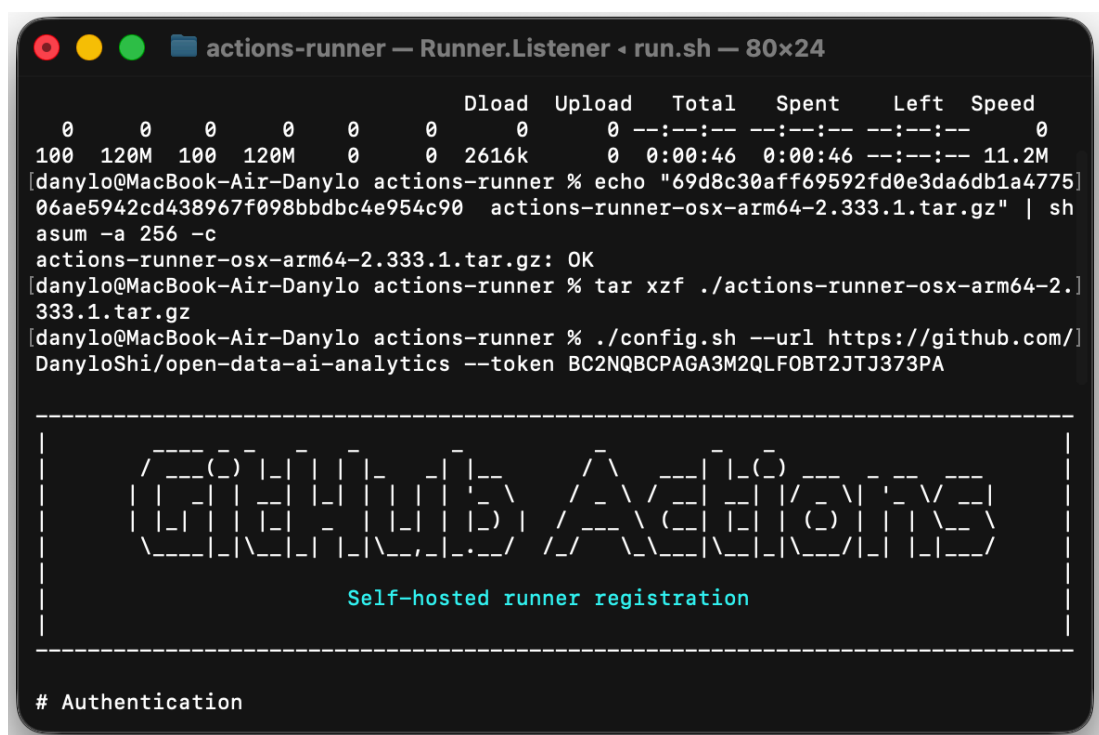


Рис. 2. Встановлення та реєстрація self-hosted runner

```

name: CI Self-Hosted

on:
  workflow_dispatch:
    inputs:
      module:
        description: "Module to run on self-hosted"
        required: true
        default: "visualization"
        type: choice
        options:
          - data_load

```

- data_quality_analysis
- data_research
- visualization

permissions:

contents: read

jobs:

run-self-hosted:

runs-on: [self-hosted, macOS, ARM64]

env:

MODULE: \${ inputs.module }

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Show Python version

run: |

python3 --version

python3 -m pip --version

- name: Create virtual environment

run: |

set -euo pipefail

python3 -m venv .venv-ci

.venv-ci/bin/python -m pip --version

- name: Install dependencies

run: |

set -euo pipefail

.venv-ci/bin/python -m pip install --upgrade pip

.venv-ci/bin/python -m pip install -r visualization/requirements.txt

.venv-ci/bin/python -m pip install -r web/requirements.txt

- name: Run selected module

run: |

set -euo pipefail

mkdir -p artifacts/\${MODULE}/work

export WORK_DIR="\$PWD/artifacts/\${MODULE}/work"

export DB_PATH="\$WORK_DIR/db/air_quality.db"

export REPORTS_DIR="\$WORK_DIR/reports"

export PLOTS_DIR="\$WORK_DIR/plots"

export CSV_PATH="\$PWD/data/dataset.csv"

.venv-ci/bin/python data_load/app.py | tee "artifacts/\${MODULE}/data_load.log"

```

        if [ "${MODULE}" != "data_load" ]; then
            .venv-ci/bin/python "${MODULE}/app.py" | tee
"artifacts/${MODULE}/${MODULE}.log"
        fi

- name: Collect generated outputs
  run: |
    set -euo pipefail
    if [ -d "artifacts/${MODULE}/work/reports" ]; then
        cp -R "artifacts/${MODULE}/work/reports" "artifacts/${MODULE}/reports"
    fi
    if [ -d "artifacts/${MODULE}/work/plots" ]; then
        cp -R "artifacts/${MODULE}/work/plots" "artifacts/${MODULE}/plots"
    fi

- name: Upload artifacts
  uses: actions/upload-artifact@v4
  with:
    name: artifacts-selfhosted-${{ inputs.module }}-${{ github.run_id }}
    path: artifacts/${{ inputs.module }}
    retention-days: 14

```

Було створено окремий workflow ci-selfhosted.yml, який запускає модуль data_load на self-hosted runner. На відміну від основного CI-конвеєра, що виконується на GitHub-hosted runner, даний workflow використовує локальну машину користувача. У процесі виконання створюється віртуальне середовище, встановлюються залежності та запускається модуль з подальшим збереженням результатів у вигляді artifacts. Це дозволяє продемонструвати можливість виконання pipeline на власному обладнанні та використання локальних ресурсів.

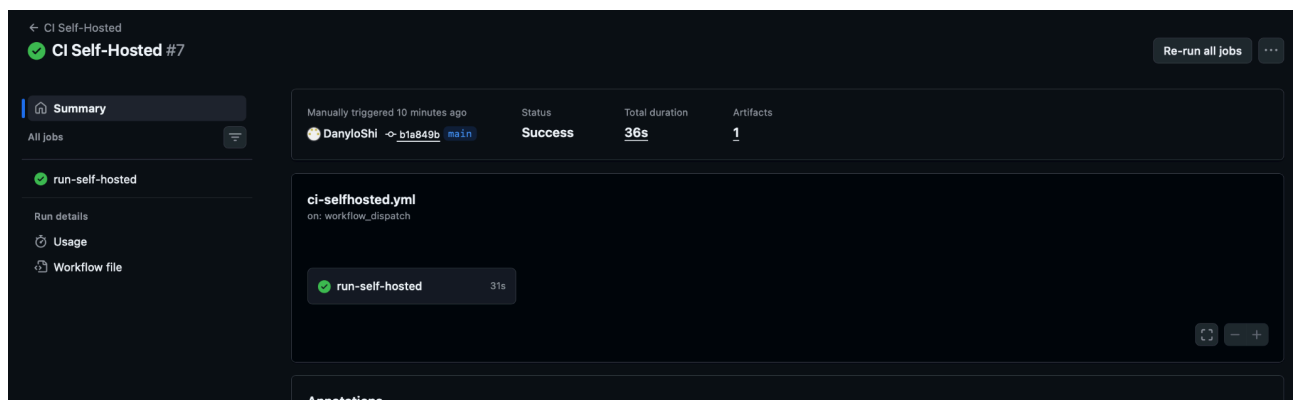


Рис. 3. Виконання CI (self-hosted runner)

У порівнянні із звичайним CI, який виконувався 25 секунд self-hosted виконався за 36 секунд, що пов'язано із встановленням бібліотек та залежностей. У випадку локального запуску ми маємо можливість не завантажувати великі датасети у workflow, проте повинні самостійно відповідати за підтримку середовища виконання

Висновок

У ході лабораторної роботи було побудовано CI/CD конвеєр із використанням GitHub Actions, реалізовано автоматичний запуск модулів при подіях push, pull request та при ручному запуску, налаштовано паралельне виконання задач за допомогою matrix-стратегії та послідовну обробку через залежності між job-ами. Було забезпечено збереження результатів між етапами за допомогою artifacts. Окремо було налаштовано self-hosted runner та виконано порівняння з GitHub-hosted runner, що дозволило зробити висновок про особливості їх використання, зокрема щодо швидкості виконання, доступу до локальних ресурсів та контролю середовища.